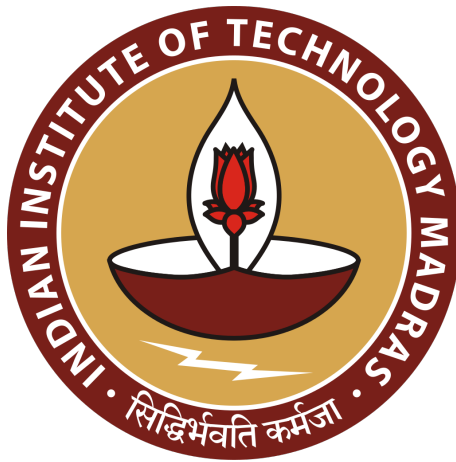


Lab Assignment - 11

Protein Secondary Structure Analysis



BT3040 - Bioinformatics

Even Semester 2026

Course Instructor: Professor Michael Gromiha

K.Varunkumar | BE23B016

Department of Biotechnology

Indian Institute of Technology, Madras

Question 1

The Python code to determine the propensity of α -helices given in Q1.

```
def propensity_alpha(seq, seq_str):
    count_alpha = 0
    propensity_scores, occurrence_helix, occurrence = {}, {}, {}

    AAs = "ACDEFGHIKLMNPQRSTVWY"
    total_helix = seq_str.count("H")

    for AA in AAs:
        propensity_scores[AA] = 0
        occurrence[AA] = seq.count(AA)
        occurrence_helix[AA] = 0

    for n in range(len(seq)):
        if seq_str[n] == "H":
            occurrence_helix[seq[n]] += 1

    for AA in propensity_scores.keys():
        AA_in_helix = occurrence_helix[AA]/occurrence[AA]
        Res_in_helix = total_helix/len(seq)

        propensity_scores[AA] = round((AA_in_helix/Res_in_helix),3)

    for AA in propensity_scores.keys():
        print(f"Score of {AA}: {propensity_scores[AA]}")
```

The output of the program is shown below:

```
Score of A: 1.551
Score of C: 1.02
Score of D: 0.0
Score of E: 0.408
Score of F: 1.02
Score of G: 1.061
Score of H: 0.875
Score of I: 1.2
Score of K: 0.0
Score of L: 1.224
Score of M: 1.531
Score of N: 1.361
Score of P: 0.227
Score of Q: 0.0
Score of R: 0.68
Score of S: 1.361
Score of T: 0.765
Score of V: 0.583
Score of W: 1.113
Score of Y: 0.583
```

Question 2

n_α = total no of residues in α -helix = 98

N = total no of residues in the whole dataset = 200

% of all residues in α -helix = $\frac{n_\alpha}{N} \times 100 = 49\%$

Propensity $\alpha(i)$ = $\frac{\% \text{ residue } i \text{ in } \alpha\text{-helix}}{\% \text{ of all residues in } \alpha\text{-helix}}$

Residue	$n_\alpha(i)$ No. of residues of i in helix	$N(i)$ No. of residues of i in the whole dataset	$\frac{n_\alpha(i)}{N(i)}$ (%)	Propensity $\alpha(i)$
G	13	25	52	1.061
A	19	25	76	1.551
V	2	7	28.57	0.583
L	12	20	60	1.224
I	10	17	58.82	1.2
P	1	9	11.11	0.227
F	10	20	50	1.02
Y	2	7	28.57	0.583
W	6	11	54.55	1.113
S	6	9	66.67	1.301
T	3	8	37.5	0.765
C	2	4	50	1.02
M	3	4	75	1.531
N	2	3	66.67	1.361
Q	0	4	0	0
D	0	5	0	0
E	1	5	20	0.408
K	0	1	0	0
R	3	9	33.33	0.690
H	3	7	42.86	0.875

Figure 2.1: Manual Propensity Calculations (verified with code results)

Question 3

The code to identify the helical and strand segments is given below:

```
def helix_identification(seq):
    helix_regions = set()
    window_size = 6
    second_window = 4

    for n in range(len(seq) - window_size + 1):
        count = 0

        cur = seq[n : n + window_size]
        cur_score = sum(helix_dict[AA] for AA in cur)

        if cur_score >= 4:

            if n + window_size < len(seq):
                count += 1
                cur = seq[n : n + window_size + count]

            else:
                helix_regions.update(set(range(n, n + window_size)))
                break

        flag = True
        while flag:
            checker = sum(chou_fasman_alpha[AA] for AA in cur[-second_window:])

            if checker < 4 :
                flag = False
                helix_regions.update(set(range(n, n + window_size + count - 1)))

            elif checker >= 4 and (n + window_size + count) >= len(seq):
                flag = False
                helix_regions.update(set(range(n, n + window_size + count)))

            # If no issues, continue extending
            else:
                count += 1
                cur = seq[n : n + window_size + count]

    contiguous_blocks, block, block_no = {}, False, 1
    start = None
    n = 0
    while n < (len(seq)):

        if (n in helix_regions) and (block == False):
            start, block = n, True

        elif (n not in helix_regions) and (block == True):
            contiguous_blocks[block_no] = (seq[start:n], (start, n-1))
```

```

        block_no += 1
        block, start = False, None

    elif (n == (len(seq)-1)) and (block == True):
        contiguous_blocks[block_no] = (seq[start:], (start, n))
        block_no += 1
        block, start = False, None

    n+=1

for ID in contiguous_blocks.keys():
    avg_propensity_score = (sum(chou_fasman_alpha[AA] for AA in
    contiguous_blocks[ID][0]))/len(contiguous_blocks[ID][0])

    print(f"The sequence {contiguous_blocks[ID][0]}, ranging from
    {contiguous_blocks[ID][1]}, is identified as a alpha-helix.
    Propensity score : {avg_propensity_score}")

return(helix_regions)

```

```

def strand_identification(seq):
    strand_regions = set()
    window_size = 5
    second_window = 3

    for n in range(len(seq) - window_size + 1):
        count = 0

        cur = seq[n : n + window_size]
        cur_score = sum(strand_dict[AA] for AA in cur)

        if cur_score >= 3:

            if n + window_size < len(seq):
                count += 1
                cur = seq[n : n + window_size + count]

            else:
                strand_regions.update(set(range(n, n + window_size)))
                break

        flag = True
        while flag:
            checker = sum(chou_fasman_beta[AA] for AA in cur[-second_window:])

            if checker < 3 :
                flag = False
                strand_regions.update(set(range(n, n + window_size + count - 1)))

            elif checker >= 3 and (n + window_size + count) >= len(seq):
                flag = False

```

```

        strand_regions.update(set(range(n, n + window_size + count)))

    else:
        count += 1
        cur = seq[n : n + window_size + count]

contiguous_blocks, block, block_no = {}, False, 1
start = None
n = 0
while n < (len(seq)):

    if (n in strand_regions) and (block == False):
        start, block = n, True

    elif (n not in strand_regions) and (block == True):
        contiguous_blocks[block_no] = (seq[start:n], (start, n-1))
        block_no += 1
        block, start = False, None

    elif (n == (len(seq)-1)) and (block == True):
        contiguous_blocks[block_no] = (seq[start:], (start, n))
        block_no += 1
        block, start = False, None

    n+=1

for ID in contiguous_blocks.keys():
    avg_propensity_score = (sum(chou_fasman_alpha[AA] for AA in
        contiguous_blocks[ID][0]))/len(contiguous_blocks[ID][0])

    print(f"The sequence {contiguous_blocks[ID][0]}, ranging from
        {contiguous_blocks[ID][1]}, is identified as a beta-strand.
        Propensity score : {avg_propensity_score}")

return(strand_regions)

```

To resolve overlapping segments:

```

def check_overlaps(overlaps, seq):
    alphas, betas = [], []
    for overlap in overlaps:
        temp_seq = seq[overlap[0]:(overlap[-1]+1)]
        alpha_score = sum(chou_fasman_alpha[AA] for AA in temp_seq)
        beta_score = sum(chou_fasman_beta[AA] for AA in temp_seq)

        if alpha_score > beta_score:
            alphas.append(overlap)
        else:
            betas.append(overlap)

    return(alphas,betas)

```

```

overlap = list(strand_regions.intersection(helix_regions))

contiguous_blocks, block, block_no = {}, False, 1
start = None
n = 0

while n < (len(overlap)):

    if block == False:
        start, block = overlap[n], True

    elif (n == (len(overlap)-1)) and (block == True):
        contiguous_blocks[block_no] = list(range(start, overlap[n]+1))
        block_no += 1
        block, start = False, None

    elif (overlap[n]+1) not in overlap:
        contiguous_blocks[block_no] = list(range(start, overlap[n]+1))
        block_no += 1
        block, start = False, None
    n+=1

op = check_overlaps(overlaps, sequence)
print(f"The regions of overlapping alpha-helices:")
for alpha in op[0]:
    print(alpha)
print("="*125)
print(f"The regions of overlapping beta-strands:")
for beta in op[1]:
    print(beta)
print("="*125)

```

The output of the above code is shown below:

```

The sequence RCELAAAMKRH, ranging from (4, 14), is identified as a alpha-helix.
The sequence WVCAAKFESNF, ranging from (27, 37), is identified as a alpha-helix.
The sequence MNAWVAWRN, ranging from (104, 112), is identified as a alpha-helix.
The sequence TDVQAWIR, ranging from (117, 124), is identified as a alpha-helix.

```

```

The sequence VFGRC, ranging from (1, 5), is identified as a beta-strand.
The sequence LAAAMKR, ranging from (7, 13), is identified as a beta-strand.
The sequence WVCAA, ranging from (27, 31), is identified as a beta-strand.
The sequence TDYGILQIN, ranging from (50, 58), is identified as a beta-strand.
The sequence AWWAAR, ranging from (106, 111), is identified as a beta-strand.
The sequence GTDVQAWIRGCRL, ranging from (116, 128), is identified as a
beta-strand.

```

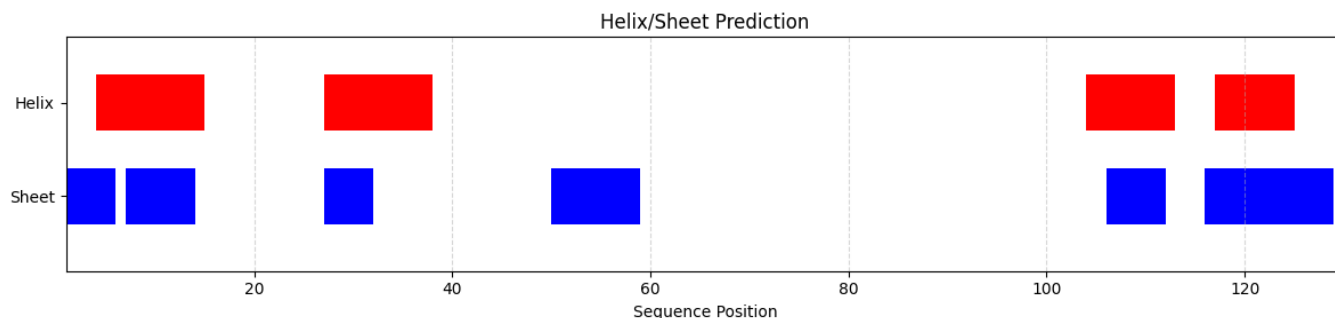


Figure 3.1: Pictorial Depiction of the Secondary Structure Predictions

The segment [4, 5] overlaps

The segment [7, 8, 9, 10, 11, 12, 13] overlaps

The segment [27, 28, 29, 30, 31] overlaps

The segment [106, 107, 108, 109, 110, 111] overlaps

The segment [117, 118, 119, 120, 121, 122, 123, 124] overlaps

=====

The regions of overlapping alpha-helices:

[7, 8, 9, 10, 11, 12, 13]

[106, 107, 108, 109, 110, 111]

=====

The regions of overlapping beta-strands:

[4, 5]

[27, 28, 29, 30, 31]

[117, 118, 119, 120, 121, 122, 123, 124]

=====

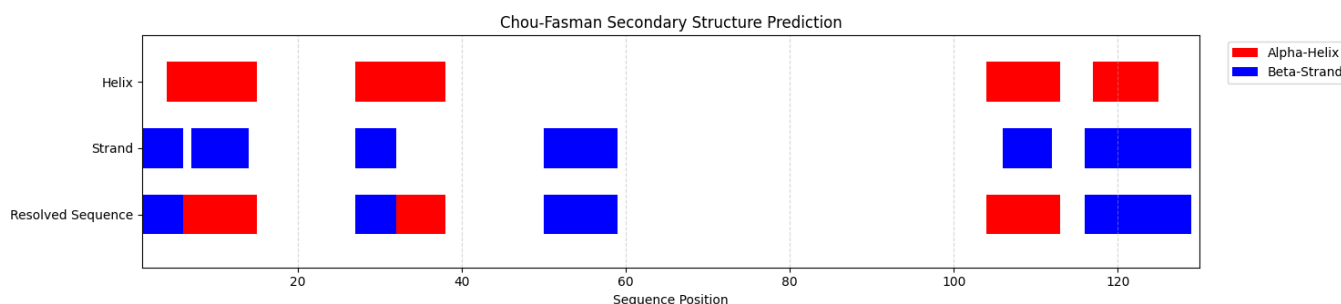


Figure 3.2: Pictorial Depiction of the Secondary Structure Predictions (Overlaps Resolved)

The extension technique employed here is unidirectional; that is, the sequence is extended only toward the right, with no extension performed on the left side.

To resolve overlaps, the sum of residue-wise propensity scores (of both α -helices and β -strands) of the overlapping region was calculated and compared to classify the segment accordingly. The average propensity score of the whole segment is not considered as it leads to issues when the overlaps are disjoint (as is the case near residues 4 to 13).

The Chou–Fasman algorithm predicts three α -helical regions and four β -strand regions, as shown in Figure 3.2.

Question 4

Verification (All zero-based indexing)

Helix - region

i) Seed-region (starts at position - 4) = RCE LAA

- ii) $\rightarrow 0 + 0 + 1 + 1 + 1 + 1 = 4$
 (extending & taking last 4 residues) = LAAA
 $\rightarrow 1.34 + 1.45 + 1.45 + 1.45 = 5.69$
- iii) (extending & taking last 4 residues) = AAAM
 $\rightarrow 1.45 + 1.45 + 1.45 + 1.2 = 5.55$
- iv) (extending & taking last 4 residues) = AAMK
 $\rightarrow 1.45 + 1.45 + 1.2 + 1.07 = 5.17$
- v) (extending & taking last 4 residues) = AMKR
 $\rightarrow 1.45 + 1.2 + 1.07 + 0.79 = 4.51$
- vi) (extending & taking last 4 residues) = MKRH
 $\rightarrow 1.2 + 1.07 + 0.79 + 1.24 = 4.3$
- vii) (extending & taking last 4 residues) = KRHG
 $\rightarrow 1.07 + 0.79 + 1.24 + 0.53 = 3.63$
 (Terminate the extension; score < 4)

Thus, RCE LAAAMKRH is identified as an α -Helix

Strand- Region

i) Seed-region (starts at position - 106) = AWVA W

- ii) $\rightarrow 0.5 + 1 + 1 + 0.5 + 1 = 4$
 (extending & taking last 3 residues) = AWR
 $\rightarrow 0.97 + 1.19 + 0.9 = 3.06$
- iii) (extending & taking last 2 residues) = WRN
 $\rightarrow 1.19 + 0.9 + 0.65 = 2.74$
 (Terminate the extension, score < 3)

Thus, AWVAWR is identified as a β -strand

Figure 4.1: Manual Verification of Secondary Structure Prediction

Link to the google drive containing the code and the result files from the different tools used in this assignment: https://drive.google.com/drive/u/0/folders/1IhVrr9M0_n0NgWF9We5_L0Q1BL1iPoGJ

Annotations have been omitted from the submitted PDF for the sake of brevity. The fully annotated version of the code is available in the code folder within the attached Google Drive.
